

MODULE 3

Growth of Functions

In algorithm analysis, we use several mathematical notations to describe how the running time or space requirement of an algorithm grows as the input size increases. The most common notations are Big O, Big Θ (Theta), and Big Ω (Omega). These notations help us classify algorithms based on their efficiency and scalability.

Big O Notation (O)

The Big O notation describes the upper bound of the growth rate of a function. It provides a worst-case scenario for the growth of the function. It is denoted as $O(f(n))$.

Example: If an algorithm's running time is $O(n^2)$, it means that the running time grows no faster than a quadratic function of the input size n .

Big Θ Notation (Θ)

Big Θ notation describes both the upper and lower bounds of the growth rate of a function. It provides a tight bound on the function's growth. It is denoted as $\Theta(f(n))$.

Example: If an algorithm's running time is $\Theta(n \log n)$, it means that the running time grows asymptotically at the same rate as $n \log n$.

Big Ω Notation (Ω)

Big Ω notation describes the lower bound of the growth rate of a function. It provides a best-case scenario for the growth of the function. It is denoted as $\Omega(f(n))$.

Example: If an algorithm's running time is $\Omega(n)$, it means that the running time grows at least linearly with the input size n .

Comparing Different Growth Rates

To understand the efficiency of algorithms, it's essential to compare their growth rates. Here are common growth rates and how they compare:

Growth Rate	Big O Notation	Description	Example

Constant	$O(1)$	The running time or space does not change with input size.	Accessing an element in an array.
Logarithmic	$O(\log n)$	The running time or space grows logarithmically.	Binary search in a sorted array.
Linear	$O(n)$	The running time or space grows linearly with input size.	Finding the maximum in an array.
Linearithmic	$O(n \log n)$	The running time or space grows as $n \log n$.	Merge sort and quicksort algorithms.
Quadratic	$O(n^2)$	The running time or space grows quadratically.	Bubble sort and insertion sort.
Cubic	$O(n^3)$	The running time or space grows cubically.	Some naive 3D matrix algorithms.
Exponential	$O(2^n)$	The running time or space grows exponentially.	Recursive solution to the traveling salesman problem.
Factorial	$O(n!)$	The running time or space grows factorially.	Generating all permutations of a set.

Comparing Growth Rates

Constant vs. Logarithmic: Constant time is faster than logarithmic time for large input sizes.

Logarithmic vs. Linear: Logarithmic time is more efficient than linear time.

Linear vs. Linearithmic: Linearithmic time (e.g., $O(n \log n)$) grows faster than linear time but is more efficient than quadratic time.

Quadratic vs. Cubic: Quadratic time grows slower than cubic time, which means algorithms with $O(n^2)$ complexity are more scalable than those with $O(n^3)$.

Exponential vs. Factorial: Exponential time grows faster than factorial time. However, both are impractical for large input sizes due to their rapid growth.

Example: Suppose we have the following values of n : 1, 10, 100, 1000, and 10,000. The following table shows the corresponding values for each function.

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$	$O(n!)$
1	1	0	1	0	1	1	2	1
10	1	2	10	20	100	1,000	1,024	3,628,800
100	1	6	100	600	10,000	1,000,000	~	~
1,000	1	9	1,000	9,000	1,000,000	1 billion	~	~
10,000	1	13	10,000	130,000	100 million	~	~	~

Observations:

Factorial time $O(n!)$: This is the most extreme case. Even for relatively small inputs like $n=10$, the function value reaches over 3.6 million. For larger values like $n=100$ or $n=1000$, the function becomes astronomically large and impractical for computation.